МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

ОТЧЕТ

по лабораторной работе №1

по дисциплине «ООП»

Тема: Введение в ООП

Студент гр. 5372 _____ Пустовалов Д.М.

Преподаватель _____ Глущенко А.Г.

Санкт-Петербург

2026

Цель работы.

Освоить основы объектно-ориентированного программирования на примере разработки игры и различных юнитов. Реализовать порождающий паттерн «Фабричный метод» для создания юнитов, а также собственный итератор для обхода юнитов на игровом поле.

Задание.

Разработать и реализовать набор классов: класс игрового поля (GameField) – контейнер для юнитов, представляющий прямоугольную сетку, набор классов юнитов (Unit) – иерархия с общим интерфейсом.

Основные теоретические положения.

ООП строится на четырех принципах: инкапсуляция, наследование, полиморфизм, абстракция.

RAII – ресурсы освобождаются деструкторами.

Правило пяти – если класс владеет ресурсом, нужно определить: конструктор копирования, деструктор, оператор присваивания копированием, конструктор перемещения, оператор присваивания перемещением.

Умные указатели – объекты, имитирующие поведение обычных указателей, но при этом сами занимаются освобождением памяти.

Фабричный метод – определяет интерфейс для создания объекта, но позволяет подклассам решать, какой класс инстанцировать.

Выполнение работы.

Реализованы классы-атрибуты такие, как Health, Armor, Attack. Каждый класс инкапсулирует одно значение и обеспечивает валидацию.

Абстрактный базовый класс Unit. Конкретные юниты реализуют clone() через std::make_unique<…>(*this).

Фабрики для создания юнитов. Конкретная фабрика, например InfantryFactory, выбирает создаваемый подтип по enum – параметру.

Игровое поле GameField (одномерный std::vector<std::unique_ptr<Unit>> размера width*height).

Итератор GameField::Iterator, выполняющий обход слева направо, сверху вниз.

В рамках работы был выбран порождающий паттерн Фабричный метод, так как задача – создавать отдельные юниты одного интерфейса.

```
Создание поля и наполнение через фабрики
  0 1 2 3 4 5 6 7
0 S S P P l l H C
1 h . . . . . . .
2 . . . . . . . .
3 . . . . . . . .
4 . . . . . . . .
5 . . . . . . . .
6 . . . . . . . .
7 . . . . . . . .
Всего юнитов: 9

Глубокое копирование
До атаки: clone HP = 120
После атаки: clone HP = 110, original HP = 120
Копия независима от оригинала.

Перемещение поля
После move: moved.getSize() = 9, field.getSize() = 0
Перемещение выполнено, исходное поле пусто.

Обход итератором
#1 Swordsman at (0,0) HP=120 ARM=15 ATK=25
#2 Swordsman at (1,0) HP=120 ARM=15 ATK=25
#3 Spearman at (2,0) HP=100 ARM=10 ATK=30
#4 Spearman at (3,0) HP=100 ARM=10 ATK=30
#5 LightArcher at (4,0) HP=70 ARM=5 ATK=20
#6 LightArcher at (5,0) HP=70 ARM=5 ATK=20
#7 HeavyArcher at (6,0) HP=90 ARM=12 ATK=28
#8 Cataphract at (7,0) HP=150 ARM=25 ATK=35
#9 HorseArcher at (0,1) HP=100 ARM=10 ATK=30
Итерация завершена. Найдено юнитов: 9
```

Рисунок 1 – Демонстрация сценариев работы.

Выводы.

В ходе лабораторной работы освоены принципы ООП. Реализован паттерн Фабричный метод. Реализован собственный итератор, обход в котором идет в порядке строк. Применено правило пяти для GameField, глубокое копирование через clone() каждого юнита. Оптимизирована передача параметров засчет использования константных ссылок, где это возможно. Продемонстрированы 4 сценария: создание и наполнение поля через фабрики, глубокое копирование, перемещение поля, обход итератором.

ПРИЛОЖЕНИЕ А

```
#pragma once
#include <string>
#include <memory>
#include "../attributes/Health.h"
#include "../attributes/Armor.h"
#include "../attributes/Attack.h"

class Unit {
protected:
    Health health_;
    Armor armor_;
    Attack attack_;
    int x_;
    int y_;

public:
    Unit(int hp, int armor, int atk)
        : health_(hp), armor_(armor), attack_(atk), x_(0), y_(0) {}

    virtual ~Unit() = default;

    virtual std::unique_ptr<Unit> clone() const = 0;
    virtual std::string getType() const = 0;
    virtual char getSymbol() const = 0;

    virtual void move(int newX, int newY) {
        x_ = newX;
        y_ = newY;
    }

    void attackUnit(Unit& target) {
        int dmg = attack_.calculateDamage(target.armor_);
        target.health_.decrease(dmg);
    }

    int getX() const { return x_; }
    int getY() const { return y_; }
    const Health& getHealth() const { return health_; }
    const Armor&  getArmor()  const { return armor_;  }
    const Attack& getAttack() const { return attack_; }
};
```

ПРИЛОЖЕНИЕ Б

```
#pragma once
#include <memory>
#include "../units/Unit.h"

class UnitFactory {
public:
    virtual ~UnitFactory() = default;
    virtual std::unique_ptr<Unit> createUnit() const = 0;
};
```

ПРИЛОЖЕНИЕ В

```
#pragma once
#include <vector>
#include <memory>
#include <stdexcept>
#include <iterator>
#include "../units/Unit.h"
#include "../factories/UnitFactory.h"

class GameField {
public:
    class Iterator {
    private:
        using BaseIt = std::vector<std::unique_ptr<Unit>>::iterator;
        BaseIt it_;
        BaseIt end_;

        void skipEmpty() {
            while (it_ != end_ && *it_ == nullptr) ++it_;
        }
    public:
        using iterator_category = std::forward_iterator_tag;
        using value_type        = Unit;
        using difference_type   = std::ptrdiff_t;
        using pointer           = Unit*;
        using reference         = Unit&;

        Iterator(BaseIt begin, BaseIt end) : it_(begin), end_(end) {
skipEmpty(); }

        Unit& operator*() const { return **it_; }
        Unit* operator->() const { return it_->get(); }

        Iterator& operator++() { ++it_; skipEmpty(); return *this; }
        Iterator operator++(int) { Iterator tmp = *this; ++(*this); return
tmp; }

        bool operator==(const Iterator& o) const { return it_ == o.it_; }
        bool operator!=(const Iterator& o) const { return it_ != o.it_; }
    };

private:
    int width_;
    int height_;
    std::size_t maxUnits_;
    std::vector<std::unique_ptr<Unit>> grid_; // PsPrPSPsPjPµCЂPSC‹PЊ
PjP°CЃCЃPёPI width*height

    std::size_t index(int x, int y) const {
        if (x < 0 || x >= width_ || y < 0 || y >= height_)
            throw std::out_of_range("Coordinates out of range");
        return static_cast<std::size_t>(y) * width_ + x;
    }

    std::size_t countUnits() const {
        std::size_t n = 0;
        for (const auto& u : grid_) if (u) ++n;
        return n;
    }
```

```
public:
    GameField(int width, int height, std::size_t maxUnits);

    GameField(const GameField& other);
    GameField& operator=(const GameField& other);

    GameField(GameField&& other) noexcept;
    GameField& operator=(GameField&& other) noexcept;

    ~GameField() = default;

    void addUnit(std::unique_ptr<Unit> unit, int x, int y);
    void removeUnit(int x, int y);
    Unit* getUnit(int x, int y);
    const Unit* getUnit(int x, int y) const;

    void fillWithFactory(const UnitFactory& factory, std::size_t count);

    int getWidth()  const { return width_;  }
    int getHeight() const { return height_; }
    std::size_t getSize()  const { return countUnits(); }
    std::size_t getMaxUnits() const { return maxUnits_; }

    void clear();

    Iterator begin() { return Iterator(grid_.begin(), grid_.end()); }
    Iterator end()   { return Iterator(grid_.end(),   grid_.end());   }
};
```

ПРИЛОЖЕНИЕ Г

